

Dilsan

Product experience and frontend

Status: Proposed work package, not a Jira assignment, client approval, or evidence of completion. Match accounts and agree the task before starting.

Shared goal and how to use this guide

The intended Sprint 1 goal is: sign in, add a supported competitor listing, request collection, and see a trustworthy observation in your own workspace. Development is planned for 28 September–11 October 2026; evaluation starts 12 October (12–18 October window). Verify course updates. No individual completion date or available hours are assumed.

First connect a page to one backend response labelled **synthetic**. This is a short learning/integration checkpoint, not the full sprint or live Amazon acceptance. Accounts, persistence, jobs and permitted source collection follow only when their dependencies are ready.

Read `docs/manual.md` sections 2–5, then the sections named below. Follow one step at a time; the later steps are a route toward the goal, not an instruction to implement the entire guide unattended. Jira owns actual status and assignments; this PDF is a dated handout.

Your deliverable and boundaries

Build a clear retailer interface and connect it to the team's API. First reviewer: Hirukshanan; Ilmam helps verify the integrated journey. Relevant scope: PB-05/07/12, FR-02-04/07/11, NFR-04/06/07. Read manual sections 6–8 and 10.

Sharplink-to-Riven design handoff

Sharplink is the selected visual reference: dark cinematic presentation, restrained large typography, square controls, fine dividers, generous spacing and a business dashboard. The preserved editable Riven prototype already adapts this direction in React/TypeScript. Ask Shewon for the reviewed prototype handoff and source; it is not currently part of the fresh repository.

Treat it as **design input and reusable UI reference**, not accepted production functionality. Its landing page/dashboard, responsive CSS, mobile navigation, loading/empty/error patterns and accessibility work can inform implementation. However, its listings, prices, histories, refresh action and collection timestamps are sample/simulated; it uses browser-local storage, has no real authentication, tenant boundary or backend integration, and used a separate Bun setup. Do not present those behaviours as completed Sprint 1 work.

Do not publish or copy the captured Sharplink mirror, branding, scripts, copy or proprietary imagery. Replace the reference hero footage and confirm font licensing and the final Riven logo before publication. Build original Riven components and content inspired by the reference; do not attempt an exact site clone. Move only understood/reviewed pieces into the team's agreed app and package workflow so there is one production frontend and one lockfile.

Avoid polishing the full marketing site while the core journey is disconnected. No charts, billing, alerts or admin screens in your first task.

First working slice: result card

1. Explain what a retailer learns from an observation: product, price, currency, availability, source, time and uncertainty. Sketch the card briefly inside this task; a separate design report is unnecessary.
2. Agree the sample response with Hirukshanan and Shewon using manual section 7. Do not rename fields independently. The sample must say synthetic, not Amazon.
3. After Ilmam verifies the minimum setup, build the card with a local sample. You may adapt an understood prototype card/style into the shared frontend rather than copying the whole prototype. Show missing price as unavailable, not zero. Keep unknown stock separate from out of stock. Show unverified matching clearly.
4. Request the agreed demo endpoint rather than retaining a second hard-coded copy. Show loading and request failure; use a retry action or an understandable recovery path.
5. Together with Hirukshanan, trace browser request, backend response and displayed fields. Ask Ilmam to run the same flow on another machine.

Checkpoint evidence: Show synthetic data from the real local endpoint, not a simulated network action. Demonstrate unknown price, unknown stock and a failed API request using controlled local examples. Check keyboard focus, readable labels and a narrow screen. Record what you actually tested.

Next slices toward the full sprint goal

Do not begin all of these at once. Select the next slice with the team after the first checkpoint.

1. **Review the visual direction.** Walk through the preserved Riven prototype with the team. Record which visual patterns/components are accepted for the product and which assets/behaviours must be replaced. Do not make migrating the whole prototype a prerequisite for the connected Sprint 1 flow.
2. **Account screens.** Once Hirukshanan defines the auth flow, build sign-in/out states against it. You may adapt the prototype's visual language, but do not simulate authentication in local storage. Check signed-out and expired-session handling. Backend authorization remains mandatory even when a button is hidden.
3. **Product and listing entry.** Use the agreed create/view endpoints and supported-source rules. Show validation, duplicate and limit messages from the API. Do not guess a client-approved quota.
4. **Collection status.** Once the job/status interface exists, connect the collection action and display pending, success and failure states. Avoid requests continuing after logout/unmount; coordinate any polling interval with B/D rather than choosing aggressive polling.
5. **Trustworthy result.** Display last success separately from last attempt, uncertainty and agreed stale status. A failed collection keeps the previous observation visible as an older result, never as a fresh success.
6. **Demonstrate.** Walk through the selected journey with another member and, when arranged, the client. Show the real supported source only after Shewon's gate passes; otherwise label live acceptance blocked.

Handoffs and explain-back

You need Hirukshanan's response/error contract and session behaviour; Shewon's field meanings and access to the reviewed prototype handoff; Ilmam's reproducible setup and job status support. Provide the UI states you need early rather than waiting for a finished backend. If a prototype component is reused, identify its source, explain the changes, remove simulated behaviour, and verify it in the shared app.

Before review, answer: Where did this displayed price come from? What happens when the request fails? Why is unknown different from zero? Which checks protect another retailer's data? Be able to change a small UI behaviour and explain it.

Review, evidence, and AI use

Before each change, explain its purpose and predict how you will check it. Implement a small change; run the check yourself; explain the result. Ask for help after a focused attempt if blocked. Never guess successful results or hide uncertainty.

Use AI as a tutor: “Read my task and the relevant manual section. Ask me to explain it. Help me plan and implement one small step, then interpret my actual check result. Do not complete the whole assignment or change shared contracts without discussion.”

For review, record: task key, what changed, why, command or walkthrough performed, actual result, limitations, material AI help, and reviewer. Use a real Jira key in your branch/PR, not the name of this guide. Done means relevant checks pass, a teammate reviews your explanation and change, and the result is integrated and demonstrated. Research tasks finish with reviewed evidence and a verdict, not invented implementation.

Stop and coordinate if an interface changes, a source requires unsupported access, a paid service is needed, or selected scope is no longer credible. Do not switch frameworks, broaden scope, or claim mock data is live. Use current repository source if this PDF is outdated.